

# ADRs — Plataforma Pin Saúde

Architecture Decision Records (MADR) · projeto greenfield · 2026-06-08

|             |                                                                            |
|-------------|----------------------------------------------------------------------------|
| Produto     | Plataforma de gestão para PJ médica (Pin Saúde)                            |
| Documento   | Conjunto fundacional de decisões de arquitetura (formato MADR)             |
| Repositório | Monorepo (Nx) · backend microsserviços com database-per-service            |
| Stack       | React · Java 17 / Spring Boot (Maven) · PostgreSQL · RabbitMQ · Playwright |
| Status      | 13 decisões Aceitas · 1 Proposta (ADR-0014) · para circulação              |

Conjunto fundacional de ADRs do projeto Pin Saúde, derivado do PRD v1.1. No repositório, cada decisão vive como um arquivo em `docs/adr/` (um ADR por decisão). As decisões fiscais e financeiras (ADR-0007 e ADR-0008) devem ser **homologadas pela contabilidade da Pin** antes do go-live, conforme o PRD (§13).

## Índice de decisões

| ADR      | Decisão                                                                         | Status   |
|----------|---------------------------------------------------------------------------------|----------|
| ADR-0001 | Adotar monorepo como estratégia de repositório                                  | Aceito   |
| ADR-0002 | Backend em microsserviços (camadas, Maven); base única no MVP, quebra posterior | Aceito   |
| ADR-0003 | Multi-tenancy por CNPJ (pooled + RLS) dentro da base de cada serviço            | Aceito   |
| ADR-0004 | PostgreSQL como banco e Flyway para versionamento de schema                     | Aceito   |
| ADR-0005 | Comunicação assíncrona com Transactional Outbox e idempotência                  | Aceito   |
| ADR-0006 | Integrações externas via Anti-Corruption Layer e escopo de integração do MVP    | Aceito   |
| ADR-0007 | Motor fiscal parametrizável e versionado por competência                        | Aceito   |
| ADR-0008 | Ledger financeiro imutável com partidas dobradas e dinheiro em inteiro          | Aceito   |
| ADR-0009 | Identidade e acesso: OAuth2/OIDC + RBAC + MFA com isolamento por tenant         | Aceito   |
| ADR-0010 | Gestão de segredos e certificados A1 por CNPJ em cofre                          | Aceito   |
| ADR-0011 | Observabilidade, trilha de auditoria e conformidade LGPD                        | Aceito   |
| ADR-0012 | Estratégia de testes e CI no monorepo (pirâmide + contratos + Playwright)       | Aceito   |
| ADR-0013 | Frontend React e contrato de API (OpenAPI) com tipos gerados no monorepo        | Aceito   |
| ADR-0014 | Modelagem de eventos de domínio e read models (CQRS na leitura)                 | Proposto |

## ADR-0001 · Adotar monorepo como estratégia de repositório

Aceito · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

### CONTEXTO

O projeto Pin Saúde começa do zero. O produto (ver PRD §3, §6) tem um backend Java 17 / Spring Boot, um frontend React (portal do médico + backoffice) e diversos contratos e bibliotecas compartilhadas (cliente de API, tipos,

regras de validação). As mudanças tendem a ser transversais: alterar um endpoint costuma exigir mudança simultânea no backend, no contrato (OpenAPI) e no frontend. Em fintech/healthtech, a rastreabilidade de "uma mudança = um PR" também ajuda auditoria (PRD §7.14).

A premissa do time é **monorepo**.

## DRIVERS DE DECISÃO

- **Commits atômicos** entre backend, contrato e frontend (mudança de contrato em um único PR).
- **Refatoração e versionamento únicos** de libs compartilhadas, sem coordenação entre repositórios.
- **Time pequeno** (3 pessoas hoje, PRD §1) — minimizar overhead de governança de múltiplos repositórios.
- **CI unificado** que entenda o grafo de dependências e construa apenas o que mudou.
- Suporte **poliglota** (JVM + JS/TS) sem exigir time de build dedicado neste estágio.

## OPÇÕES CONSIDERADAS

1. **Polyrepo** (um repositório por app/serviço) — descartado: mudança de contrato exige PRs coordenados em vários repos; alto custo de governança para time pequeno.
2. **Monorepo com apenas workspaces de pacote** (pnpm/npm workspaces, sem orquestrador) — insuficiente: não orquestra tarefas nem faz cache; não cobre o lado JVM.
3. **Monorepo com orquestrador leve (Nx ou Turborepo)** sobre Maven (JVM) + pnpm (JS/TS).
4. **Monorepo com build system hermético (Bazel / Pants / Buck2)** — poderoso e determinístico, porém com curva e custo de manutenção altos; exige time de build dedicado. Inadequado para o estágio de MVP.

## DECISÃO

Adotar **monorepo único** com a seguinte composição:

- **Orquestrador: Nx** — tem integração nativa com React e suporte a **Maven** (JVM) via plugin/targets, grafo de dependências, cache local/remoto, build do "affected" e enforcement de fronteiras entre projetos.
- **Backend: microsserviços Spring Boot** construídos com **Maven** (reactor multi-módulo; um serviço por bounded context), conforme ADR-0002.
- **Frontend: workspace pnpm** para React e libs JS/TS.
- **Bazel/Pants** ficam registrados como caminho de evolução caso o repo cresça a ponto de o cache/precisão do Nx não bastar (revisitar em ADR futuro).

## Estrutura proposta

```
pin-saude/
├─ apps/
│   └─ web/                # React SPA (portal + backoffice)
├─ services/               # microsserviços Spring Boot (Maven), um por contexto
│   ├── fiscal/
│   ├── faturamento/
│   ├── ledger/
│   ├── repasse/
│   ├── onboarding/
│   └─ gestao/
├─ gateway/                # API Gateway / BFF (ADR-0002)
├─ libs/
│   └─ frontend/           # libs JS/TS (ui, api-client gerado, design-system)
├─ contracts/              # OpenAPI por serviço + schemas (contratos REST e de eventos)
├─ tools/                  # geradores, scripts de CI, hooks
├─ docs/adr/               # estes ADRs
├─ nx.json
└─ pnpm-workspace.yaml
```

```
| pom.xml  
└ ...
```

```
# POM-pai (reactor Maven)
```

## CONSEQUÊNCIAS

**Positivas** - Um `git clone`, uma fonte da verdade; mudanças transversais em um PR. - CI por "affected" reduz tempo de pipeline conforme o repo cresce. - Fronteiras de módulo verificáveis (Nx module boundaries) reforçam a arquitetura do ADR-0002.

**Negativas / riscos** - Repositório e CI precisam de disciplina: sem cache/affected, o monorepo fica mais lento que polyrepo. Mitigação: configurar cache (e remoto) desde o dia 1. - Acoplamento accidental entre módulos é mais fácil — mitigado por boundaries e CODEOWNERS por pasta. - Controle de acesso é por repositório; se no futuro houver necessidade de isolamento forte de partes do código, reavaliar.

## REFERÊNCIAS

PRD §3 (Escopo), §6 (Arquitetura). Relacionado: ADR-0002, ADR-0012, ADR-0013.

# ADR-0002 · Backend em microsserviços (camadas, Maven); base única no MVP, quebra posterior

Aceito · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

## CONTEXTO

O domínio tem limites claros (faturamento/emissão, fiscal, ledger, recebimento/ conciliação, repasse, onboarding, gestão — PRD §8) e forte dependência de terceiros (agregador fiscal, Clicksign, Conta Azul — PRD §9). O time adota **microsserviços desde o início**, com **arquitetura em camadas tradicional** por serviço e **Maven** como build.

Para a **persistência no MVP**, o time avaliou que **uma base/schema por serviço (database-per-service) é dispendioso demais agora**: para a base-alvo (~5.000 médicos) uma **única base PostgreSQL** bem indexada atende sem stress, com muito menos custo e operação. A quebra em bases por serviço fica para quando o crescimento justificar.

Decisão do time (revisão 2026-06-08). Substitui o monólito modular hexagonal/ Gradle (rev. 0) e o database-per-service no MVP (rev. 2): **MVP com base única**, mantendo a fronteira lógica para que a quebra futura seja barata.

## DRIVERS DE DECISÃO

- **Custo e operação enxutos no MVP** (um cluster, um backup, menos infra) para time de 3 pessoas.
- Uma única base PostgreSQL **comporta a base-alvo (~5.000 médicos)** com folga.
- Limites de domínio nítidos e deploy independente por serviço.
- **Preservar a opção de quebrar depois** sem reescrever (fronteira lógica desde já).
- Padronização de build com **Maven**; simplicidade interna por serviço (camadas).

## OPÇÕES CONSIDERADAS

1. **Database-per-service desde o MVP** — máximo isolamento; descartado: custo/operação de N bases não se justifica no estágio atual.
2. **Base única e schema único compartilhado por todos (acesso livre às tabelas)** — mais simples, porém vira **monólito distribuído** e encarece a quebra futura. Descartado.
3. **Base única com um schema por serviço (ownership lógico) + disciplina de fronteira** — escolhido: simples e barato agora, sem fechar a porta da quebra.

## DECISÃO

Adotar **microsserviços desde o MVP**, cada um Spring Boot com **Maven**:

- **Um serviço por bounded context** (ex.: `fiscal`, `faturamento`, `ledger`, `repasse`, `onboarding`, `gestao`), em **arquitetura em camadas** (`controller` → `service` → `repository` + DTOs/mapeamento).
- **Persistência no MVP: uma única base PostgreSQL**, com **um schema por serviço** (ownership lógico). Para manter a quebra futura barata, vale a **disciplina de fronteira**, mesmo sendo fisicamente possível burlá-la:
- cada serviço **só acessa o seu schema** (idealmente com **usuário/credencial de banco próprio por serviço**, sem GRANT nos schemas alheios);
- **sem JOIN nem FK cruzando bounded contexts**;
- **sem transação abrangendo dados de mais de um serviço** (nada de "aproveitar" a base única para transação cross-serviço).
- **Dados de outro contexto** vêm por **API REST** (ADR-0013) ou **eventos** (RabbitMQ + Outbox — ADR-0005), **não** por consulta direta ao schema alheio.
- **Consistência cross-service** por **saga + outbox + idempotência** (ADR-0005/0008); **sem 2PC**.
- **Consolidações que cruzam serviços** (apuração, DRE, teto — PRD §7.9) ficam no **read model do serviço de gestão** (ADR-0014).
- **Multi-tenancy** (`cnpj_id` + RLS) na base única do MVP (ADR-0003).

- **Gatilho de quebra** (→ **database-per-service**): promover o schema de um serviço a base própria quando aparecer **contenção de I/O/locks**, necessidade de **escala ou janela de manutenção independente**, **blast radius** inaceitável, ou **crescimento do time** por serviço. Como o schema já é isolado, a promoção é de baixo custo (ADR-0004 mantém migrações por serviço).
- **Build Maven** (reactor com POM-pai; Nx orquestra alvos e cache — ADR-0001).
- **API Gateway / BFF** na borda para o frontend — detalhar em ADR de gateway.

## CONSEQUÊNCIAS

**Positivas - Custo e operação baixos no MVP:** um cluster/backup, menos infra para 3 pessoas. - Atende a base-alvo (~5.000 médicos) sem stress. - Fronteira lógica preservada → **quebra futura barata** (schema vira base).

**Negativas / riscos - Risco de virar "monólito distribuído"** se a disciplina de fronteira não for seguida (acesso cross-schema, FK/JOIN, transação cross-serviço). Mitigação: **usuário de banco por serviço** (impede acesso indevido no nível do Postgres), revisão de código e testes; tratar o **gatilho de quebra** como compromisso real, não dívida indefinida. - **Ponto único de falha/contención** na base única — monitorar I/O, locks e conexões; promover serviço(s) a base própria quando o gatilho disparar. - **Consistência eventual** entre serviços (não ACID) → saga/outbox/idempotência; estados claros na UX. - **Overhead de microsserviços para time de 3** — poucos serviços no início; CI/CD e observabilidade desde o dia 1 (ADR-0011/0012).

## REFERÊNCIAS

PRD §6, §7.9, §8, §9, §11. Relacionado: ADR-0001, ADR-0003, ADR-0004, ADR-0005, ADR-0008, ADR-0011, ADR-0013, ADR-0014.

# ADR-0003 · Multi-tenancy por CNPJ (pooled + RLS) dentro da base de cada serviço

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

## CONTEXTO

A plataforma nasce multi-empresa: vários CNPJs, com roteamento de médicos por capacidade de faturamento e teto do Lucro Presumido por CNPJ (PRD §5.1, §7.9c). O isolamento de dados por CNPJ é requisito transversal e de auditoria/LGPD (PRD §4, §7.14, §11). Um usuário pode acessar 1..N empresas, e um médico pode estar vinculado a N CNPJs (PRD §5.1, §7.1).

Com a **base única do MVP** (ADR-0002), esta decisão trata de **como isolar os tenants (CNPJs)** dentro dessa base — é ortogonal e complementar ao isolamento lógico por serviço (schema por serviço). Quando a base for quebrada por serviço, o mesmo modelo de tenancy continua valendo, agora por base de serviço.

## DRIVERS DE DECISÃO

- Isolamento forte e auditável de dados por tenant.
- Operação simples para time pequeno.
- Escala até milhares de médicos e dezenas de CNPJs (PRD §11).
- Consolidações de gestão **cross-tenant** (apuração e teto fiscal por CNPJ).

## OPÇÕES CONSIDERADAS

(Dentro do **schema de cada serviço**, na base única do MVP — ADR-0002:) 1. **Banco por tenant** — isolamento máximo; custo operacional e de migração alto. Inadequado ao estágio. 2. **Schema por tenant** — bom isolamento; migrações por schema viram gargalo com muitos CNPJs. 3. **Pooled: tabelas com tenant\_id (cnpj\_id) + Row-Level Security (RLS) do PostgreSQL** — isolamento aplicado no banco, operação simples.

## DECISÃO

Adotar o modelo **pooled** dentro do schema de cada serviço (base única do MVP — ADR-0002): `cnpj_id` em toda tabela multi-tenant e **RLS no PostgreSQL** como rede de segurança:

- O tenant atual é resolvido do contexto de autenticação e propagado (ex.: `SET app.current_tenant`).
- Políticas de RLS garantem que mesmo um bug na aplicação não vaze dados entre tenants.
- **Consolidações cross-tenant** (apuração, DRE, monitor de teto — PRD §7.9) ficam no **read model do serviço de gestão** (alimentado por eventos — ADR-0014), com um contexto explícito autorizado a leituras cross-tenant. Mesmo com base única, evita-se consultar os schemas alheios diretamente, preservando a fronteira (ADR-0002).
- Caminho de evolução: um tenant de altíssimo volume pode ser promovido a schema/ base própria sem mudar o modelo de domínio.

## CONSEQUÊNCIAS

**Positivas** - Isolamento defendido na camada mais baixa (banco), reforçando o RBAC da aplicação (ADR-0009). - Operação enxuta (base única no MVP). - Isolamento de tenant (RLS) e de serviço (schema/ownership) compõem defesa em profundidade.

**Negativas / riscos** - Esquecer `cnpj_id` em uma query é o erro clássico — mitigado por RLS + revisão + testes de isolamento (ADR-0012). - RLS adiciona leve overhead e exige cuidado com índices (sempre compostos com `cnpj_id`). - Apuração/teto via **read model por eventos** (ADR-0014) — custo coberto lá.

## REFERÊNCIAS

PRD §4, §5.1, §7.1, §7.9, §11, §13 (N3). Relacionado: ADR-0002, ADR-0004, ADR-0005, ADR-0009, ADR-0014.

# ADR-0004 · PostgreSQL como banco e Flyway para versionamento de schema

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

## CONTEXTO

O PRD define PostgreSQL (§6). O domínio é relacional e transacional (notas, ledger, apuração — PRD §8) e exige retenção fiscal de 5 anos e trilhas de auditoria (PRD §7.14, §11). Com a **base única do MVP** (ADR-0002) organizada em **um schema por serviço**, cada serviço evolui o **seu schema** de forma segura, reproduzível e **independente** — o que também mantém barata a futura quebra em bases por serviço.

## DRIVERS DE DECISÃO

- Integridade transacional e consistência (operações financeiras).
- Versionamento de schema **por serviço**, reproduzível e auditável em todos os ambientes.
- Evolução de schema independente por serviço (sem migração global acoplada).
- Recursos avançados (RLS para multi-tenancy — ADR-0003; JSONB para payloads de integração).

## OPÇÕES CONSIDERADAS

1. **Hibernate ddl-auto (update)** — proibido fora de protótipo: imprevisível e perigoso em produção.
2. **Liquibase** — robusto, formato XML/YAML/SQL; mais verboso.
3. **Flyway** — migrações versionadas em SQL puro, simples e previsíveis; ótima integração com Spring Boot.

## DECISÃO

- **PostgreSQL** como banco relacional. No **MVP**, **uma única base** (ADR-0002) com **um schema por serviço**; cada serviço migra **apenas o seu schema**.
- **Flyway por serviço**: cada serviço tem seu próprio conjunto de migrações **SQL versionadas e imutáveis** (`V<versão>__descricao.sql`), seu próprio histórico (tabela de schema history própria) e aponta para o **seu schema**; nenhuma alteração de schema fora de migração.
- `ddl-auto=validate` em runtime (a aplicação nunca altera schema).
- Migrações vivem no monorepo, **junto do código do serviço** que as exige.
- **Quebra futura**: como cada serviço já versiona o seu schema isoladamente, ao promover um serviço a base própria as migrações vão junto, **sem reescrita**.

## CONSEQUÊNCIAS

**Positivas** - Schema reproduzível e auditável **por serviço**; evolução independente. - Rollback planejado por migração de compensação. - SQL puro deixa explícitas as policies de RLS e índices compostos por tenant. - Migração de "base única" → "base por serviço" sem retrabalho de scripts.

**Negativas / riscos** - Disciplina: migração aplicada nunca é editada (sempre nova). Mitigação: validação no CI. - Migrações de grande volume exigem estratégia (online/expand-contract) — registrar quando surgir.

## REFERÊNCIAS

PRD §6, §8, §11. Relacionado: ADR-0002, ADR-0003, ADR-0008.

# ADR-0005 · Comunicação assíncrona com Transactional Outbox e idempotência

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

## CONTEXTO

Emissão fiscal, conciliação e repasse dependem de terceiros e precisam de filas com **retry e idempotência** (PRD §6, §11). Há picos de emissão nos dias 1–7 de cada mês (PRD §11). Eventos disparam efeitos colaterais (ex.: recebimento → crédito no ledger → cálculo de líquido → repasse — PRD §10) que não podem ser perdidos nem duplicados (risco financeiro).

## DRIVERS DE DECISÃO

- Não perder eventos nem duplicar repasses/emissões.
- Resiliência a indisponibilidade de terceiros (PRD §14).
- Absorver picos mensais sem degradar a experiência síncrona.
- Manter operação simples no MVP.

## OPÇÕES CONSIDERADAS

1. **Publicar direto no broker dentro da transação de negócio** — sofre de dual-write (banco e broker podem divergir).
2. **Transactional Outbox** — evento gravado na mesma transação do dado; um relay publica de forma assíncrona e confiável.
3. **Broker:** RabbitMQ (simples, roteamento rico) vs Kafka (alto throughput/retenção, mais operação) vs fila gerenciada do provedor de nuvem.

## DECISÃO

- Adotar **Transactional Outbox**: o evento é persistido na mesma transação que altera o estado de negócio; um relay (poller/CDC) publica para o broker.
- **Idempotência ponta a ponta**: chaves de idempotência por comando externo (emissão, PIX) e consumidores idempotentes (dedupe por `message_id`).
- **Broker: RabbitMQ** (decidido pelo time) — roteamento rico (exchanges/ routing keys), filas com retry/DLQ e operação simples; adequado ao volume do MVP e à comunicação assíncrona entre serviços (ADR-0002). Kafka fica como evolução caso surja necessidade de event streaming/retenção longa.
- Filas com **retry com backoff** e **dead-letter queue**; nenhuma operação financeira sem chave de idempotência.

## CONSEQUÊNCIAS

**Positivas** - Consistência entre estado e eventos; reprocessamento seguro. - Picos absorvidos pelas filas; UI responde rápido.

**Negativas / riscos** - Consistência eventual entre serviços — aceitável, mas exige UX/estados claros (ex.: "emitindo", "processando") e **sagas** para fluxos que cruzam serviços (ADR-0002/0008). - Outbox exige relay e limpeza/particionamento da tabela. Mitigação: job de manutenção + métricas de lag.

## REFERÊNCIAS

PRD §6, §7.4, §7.7, §7.8, §10, §11, §14. Relacionado: ADR-0002, ADR-0006, ADR-0008.

# ADR-0006 · Integrações externas via Anti-Corruption Layer e escopo de integração do MVP

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

## CONTEXTO

A operação depende de terceiros heterogêneos: agregador fiscal / Ambiente Nacional NFS-e, Clicksign, Conta Azul, consulta de CNPJ e, no desenho original, banco (Inter/BTG via Open Finance) e BaaS para PIX (PRD §9). O PRD pede integrações "isoladas atrás de adapters, para permitir troca de fornecedor" (§6) e fallback de emissão manual (RF-NF-09).

**Decisão de escopo do time (revisão 2026-06-08):** no MVP não haverá integração com bancos. A conciliação será feita por importação de dados (arquivo de extrato) e o PIX será executado manualmente por um colaborador. Automação bancária (Open Finance) e PIX via BaaS passam para a **Fase 2**. (Refina o PRD §9 / RF-REC-01 / RF-REP-02, que previam integração bancária já na Fase 1.)

## DRIVERS DE DECISÃO

- Trocar de fornecedor (ex.: agregador fiscal) sem tocar no domínio.
- Resiliência a indisponibilidade (PRD §14) e fallback operacional (RF-NF-09).
- **Reduzir escopo e risco do MVP:** evitar a complexidade/homologação de integração bancária e PIX automatizado no primeiro release.
- Modelos externos não devem contaminar o modelo de domínio.

## OPÇÕES CONSIDERADAS

1. **Chamar SDKs/clients do fornecedor direto no domínio** — acopla o núcleo ao fornecedor; troca custosa.
2. **Anti-Corruption Layer (ACL):** cada integração é um **adapter** atrás de um **port** do domínio. Para banco/PIX, o port recebe um **adapter manual/importação** no MVP e um **adapter de API** na Fase 2, sem mudar o domínio.

## DECISÃO

- Cada integração externa fica atrás de um **port** (interface) com **adapters** intercambiáveis (ACL). Padrões de resiliência por adapter: **timeouts**, **retry com backoff**, **circuit breaker**, **bulkhead** (ex.: Resilience4j).
- **Integrações ativas no MVP (Fase 1):** agregador fiscal/NFS-e (RF-NF-03), Clicksign, Conta Azul, consulta de CNPJ. Chamadas que alteram estado externo (emissão) usam **idempotência** (ADR-0005). **Fallback** de emissão manual previsto (RF-NF-09).
- **Conciliação no MVP — por importação:** `ConciliacaoPort` com `ImportacaoExtratoAdapter` (upload de arquivo CSV/OFX do extrato). O matching segue RF-REC-02 (valor + tomador + data + confirmação humana) e dá baixa/credita o ledger (ADR-0008). **Fase 2:** `OpenFinanceAdapter` (Inter/BTG) substitui o adapter de importação, sem mudar o domínio.
- **Repasse/PIX no MVP — manual:** `RepassePort` com `RepasseManualAdapter`. O sistema **calcula o líquido (85%), gera a lista/instrução de repasses a pagar**, o colaborador executa o PIX no app do banco e **registra comprovante e status** na plataforma. Mesmo manual, o repasse é **idempotente** (marcação de "pago" por repasse) para **impedir pagamento em duplicidade**, e fica sob aprovação (RF-REP-04) e trilha de auditoria (ADR-0011). **Fase 2:** `BaaSPixAdapter` automatiza a execução, incluindo split acima de R\$ 40 mil (RF-REP-03).
- Contratos de fornecedor cobertos por **testes de contrato** (ADR-0012).

## CONSEQUÊNCIAS

**Positivas** - Troca de fornecedor isolada ao adapter; **migração MVP → Fase 2 sem mexer no domínio** (só troca o adapter). - **Menor risco e escopo no MVP** (sem homologação bancária nem PIX automatizado). - Falhas de terceiros não derrubam o núcleo; degradação controlada.

**Negativas / riscos** - **PIX manual é gargalo operacional e risco financeiro** (erro de digitação, pagamento duplicado, atraso no SLA). Mitigação: o sistema controla a worklist, exige conferência/aprovação, registra comprovante e

bloqueia duplo pagamento por idempotência; priorizar a automação (Fase 2) conforme o volume crescer. -

**Conciliação por importação** depende de um passo manual de upload e da qualidade do arquivo do banco; manter o matching assistido (RF-REC-02). - Mais código (port + adapter + tradução). Aceitável dado o risco de fornecedor.

## REFERÊNCIAS

PRD §6, §9, §10, §14, RF-NF-03, RF-NF-09, RF-REC-01..03, RF-REP-01..05. Relacionado: ADR-0005, ADR-0008, ADR-0010.

## ADR-0007 · Motor fiscal parametrizável e versionado por competência

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

### CONTEXTO

O PRD é enfático: nenhuma alíquota/base pode estar "chumbada" no código; tudo é **parametrizável por competência** (RF-FISC-04, §11). As regras incluem equiparação hospitalar por CNAE+serviço, destaque padronizado, presunções reduzidas/cheias e casos especiais — como a **nota de paciente CPF com equiparação emitida com impostos zerados** (PRD §5.2, RF-NF-02/RF-FISC-02). Há ainda a **transição IBS/CBS** a partir de 2027 (PRD §5.2, §13, §14), que mudará o regime e precisa conviver com as regras atuais por período.

### DRIVERS DE DECISÃO

- Mudança fiscal é **regra de negócio**, não release de código.
- Reprocessar/auditar uma competência passada com as regras vigentes **naquela** data.
- Suportar variantes (nota zerada para CPF + equiparação) e a transição IBS/CBS.
- Homologação das fórmulas pela contabilidade (PRD §13).

### OPÇÕES CONSIDERADAS

1. **Alíquotas em `application.yml` /constantes** — descartado: muda regra via deploy; sem histórico por competência.
2. **Tabela de parâmetros simples (chave → valor)** — melhor, mas não resolve regras condicionais nem vigência.
3. **Tabelas de regras versionadas por vigência + motor de regras** — parâmetros e regras com `vigencia_inicio/fim`, selecionados pela competência do fato gerador; estratégias plugáveis por tipo de tomador/serviço.

### DECISÃO

- Modelar **regras e alíquotas como dados versionados por vigência** (não código).
- O motor seleciona a regra pela **competência do fato gerador** (não pela data atual) → reprocessamento histórico fiel.
- Estratégias por caso: tomador PJ com retenção, tomador PF, **PF/CPF com equiparação (destaque zerado, tributo apurado à parte — RF-FISC-02)**, e o novo regime **IBS/CBS** (NBS 200029 / Anexo III / redução de 60% / início 2027) como conjunto de regras com vigência própria.
- Separar **destaque na nota de apuração mensal** (são coisas distintas — ver §5.2 e ADR-0008/§7.9 do PRD).
- Cada cálculo grava **trilha** (qual regra/versão foi aplicada) para auditoria.
- Fórmulas marcadas como **a homologar** pela contabilidade antes do go-live.

### CONSEQUÊNCIAS

**Positivas** - Mudança fiscal sem deploy; histórico e auditoria por competência. - Transição IBS/CBS modelada como dados, reduzindo retrabalho (mitiga risco do PRD §14).

**Negativas / riscos** - Motor de regras é complexo e crítico — exige cobertura de testes alta e validação contábil.

Mitigação: testes baseados em exemplos homologados + PBT (property-based) para invariantes (ex.: médico sempre recebe 85%).

### REFERÊNCIAS

PRD §5.2, §5.3, §7.5, §7.9, §13, §14, RF-FISC-01..04, RF-NF-02. Relacionado: ADR-0008, ADR-0012.

# ADR-0008 · Ledger financeiro imutável com partidas dobradas e dinheiro em inteiro

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

## CONTEXTO

O coração financeiro é o ledger por médico: crédito da nota, retenções, ISS, taxa administrativa, repasse e ajustes, em **regime de caixa** (PRD §7.6). O repasse é distribuição de lucro e deve conciliar com a contabilidade oficial (RF-LED-04). Erros aqui têm impacto direto em dinheiro de terceiros.

## DRIVERS DE DECISÃO

- Correção financeira e **auditabilidade total** (PRD §7.14).
- Conciliação com a contabilidade (RF-LED-04).
- Nunca perder/duplicar lançamentos; reprocessamento seguro.
- Evitar erros de arredondamento.

## OPÇÕES CONSIDERADAS

1. **Tabela de saldo mutável (update no saldo)** — descartado: perde histórico, difícil auditar, propenso a corrida.
2. **Lançamentos imutáveis (append-only) + saldo derivado** — histórico completo; saldo é projeção dos lançamentos.
3. **Partidas dobradas (double-entry)** — cada movimento tem débito e crédito equilibrados; padrão contábil, casa naturalmente com a contabilidade.

## DECISÃO

- **Ledger append-only**: lançamentos são **imutáveis**; correções são **estornos + novo lançamento**, nunca edição.
- Adotar **partidas dobradas**: facilita a conciliação com a contabilidade (RF-LED-04) e torna invariantes verificáveis.
- **Dinheiro sempre em inteiro (centavos)** ou `BigDecimal` com escala e arredondamento explícitos — **nunca `double` / `float`**.
- Saldo é **derivado** (projeção) dos lançamentos; idempotência por origem (ADR-0005) impede duplicidade.
- Toda operação registra trilha de quem/quando/origem (PRD §7.14).

## CONSEQUÊNCIAS

**Positivas** - Auditoria e conciliação naturais; reprocessamento e replay seguros. - Invariantes testáveis (ex.: soma débitos = créditos; repasse = 85% do bruto).

**Negativas / riscos** - Mais lançamentos e necessidade de projeções/materialização para performance. Mitigação: views materializadas/snapshots de saldo. - Curva conceitual de double-entry para quem não é da contabilidade. Mitigação: documentação + revisão contábil.

## REFERÊNCIAS

PRD §5.3, §7.6, §7.7, §7.8, §7.14, RF-LED-01..04. Relacionado: ADR-0004, ADR-0005, ADR-0007.

# ADR-0009 · Identidade e acesso: OAuth2/OIDC + RBAC + MFA com isolamento por tenant

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

## CONTEXTO

O PRD pede OAuth2/OIDC, MFA para perfis administrativos, RBAC por perfil e isolamento multi-tenant por CNPJ (PRD §4, §6, §7.1, §11). Há perfis distintos (médico, operação, financeiro, contábil, gestão) e um usuário pode acessar 1..N empresas.

## DRIVERS DE DECISÃO

- Segurança de dados financeiros/fiscais e LGPD.
- RBAC por perfil + isolamento por tenant (defesa em profundidade com ADR-0003).
- Não construir gestão de identidade do zero.
- MFA obrigatório para perfis administrativos.

## OPÇÕES CONSIDERADAS

1. **Autenticação própria (usuário/senha caseiro)** — descartado: risco de segurança e reinvenção.
2. **IdP gerenciado (ex.: Auth0/Cognito/equivalente)** — rápido, menos operação, custo por uso.
3. **Keycloak self-hosted (OIDC)** — open-source, controle total, suporta RBAC e MFA; mais operação.

## DECISÃO

- Padrão **OAuth2/OIDC**; aplicação valida tokens (resource server), nunca guarda senha.
- **MFA obrigatório** para Operação, Financeiro, Contábil e Gestão (RF-AUTH-02).
- **RBAC** por perfil (PRD §4); autorização também valida o **tenant** do recurso (claim de CNPJ ↔ `cnpj_id`), em conjunto com a RLS do ADR-0003.
- Escolha do IdP (**Keycloak** self-hosted vs IdP gerenciado) decidida em ADR dedicado conforme nuvem/custo; o código depende de **OIDC padrão**, não do fornecedor.
- Operação sensível (troca de dados bancários — RF-ONB-08) exige reconfirmação/step-up.

## CONSEQUÊNCIAS

**Positivas** - Segurança madura sem reinvenção; SSO e MFA prontos. - Autorização em camadas (token + RBAC + RLS).

**Negativas / riscos** - Dependência de IdP — isolada por OIDC padrão. - Mapear claims → tenant/perfil com cuidado; testes de autorização por tenant (ADR-0012).

## REFERÊNCIAS

PRD §4, §6, §7.1, §7.3 (RF-ONB-08), §11, RF-AUTH-01..04. Relacionado: ADR-0003, ADR-0010.

## ADR-0010 · Gestão de segredos e certificados A1 por CNPJ em cofre

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

### CONTEXTO

Cada CNPJ tem um **certificado A1** usado para assinar NFS-e, além de credenciais de banco, BaaS, agregador e IdP (PRD §6, §7.4, RF-CAD-02). O PRD exige A1 em cofre de segredos (secrets/HSM) e criptografia de CPF/dados bancários (§6, §11). É material altamente sensível e regulado.

### DRIVERS DE DECISÃO

- Proteger chaves privadas (A1) e credenciais — nunca em repositório ou variável de ambiente em claro.
- Um A1 por CNPJ, com rotação e expiração.
- Auditoria de acesso a segredos (PRD §7.14).

### OPÇÕES CONSIDERADAS

1. **Segredos em arquivo/env/banco em claro** — inaceitável.
2. **Secrets manager da nuvem** (ex.: AWS Secrets Manager / GCP Secret Manager / Azure Key Vault) — gerenciado, integra com IAM.
3. **HashiCorp Vault** — agnóstico de nuvem, suporta engine de PKI e dynamic secrets; mais operação.

### DECISÃO

- Todos os segredos (incluindo **A1 por CNPJ**) ficam em **cofre dedicado** (Secrets Manager gerenciado ou Vault — decidir conforme nuvem em ADR próprio).
- A1 carregado **em memória sob demanda** para assinatura; nunca persistido em disco da aplicação.
- **Rotação** e monitoramento de **expiração** de certificados, com alerta antecipado à operação.
- **Criptografia** de CPF e dados bancários em repouso (ADR-0004) e acesso a segredos **auditado**.
- Acesso a segredos por **identidade da aplicação** (IAM/role), com menor privilégio por ambiente.

### CONSEQUÊNCIAS

**Positivas** - Material sensível protegido e auditável; rotação sem deploy. - Isolamento por CNPJ alinhado ao multi-tenant (ADR-0003).

**Negativas / riscos** - Dependência do cofre na disponibilidade da emissão — mitigar com cache seguro de curta duração e tratamento de indisponibilidade (ADR-0006). - Gestão do ciclo de vida dos A1 (vencimento) é responsabilidade operacional — cobrir com alertas.

### REFERÊNCIAS

PRD §6, §7.4, §11, RF-CAD-02. Relacionado: ADR-0004, ADR-0006, ADR-0009.

## ADR-0011 · Observabilidade, trilha de auditoria e conformidade LGPD

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

### CONTEXTO

O PRD exige observabilidade e auditoria ponta a ponta (§6), trilha de toda ação fiscal/financeira (§7.14), retenção fiscal de 5 anos e tratamento de dados pessoais/paciente conforme LGPD (§7.14, §11). Processamento assíncrono (ADR-0005) torna o rastreo distribuído indispensável.

### DRIVERS DE DECISÃO

- Diagnosticar problemas em fluxos assíncronos e com terceiros.
- **Auditoria imutável** de quem informou produção, emitiu, aprovou e repassou.
- LGPD: minimização, retenção e proteção de dados pessoais.

### OPÇÕES CONSIDERADAS

1. **Apenas logs de aplicação** — insuficiente para fluxos assíncronos e auditoria de negócio.
2. **Observabilidade via OpenTelemetry** (traces/métricas/logs correlacionados) + **trilha de auditoria de negócio separada** e imutável.

### DECISÃO

- **OpenTelemetry** como padrão de instrumentação (traces, métricas, logs correlacionados por `trace_id / correlation_id`), exportando para o backend de observabilidade escolhido (decisão de fornecedor em ADR próprio).
- **Trilha de auditoria de negócio é separada dos logs**, append-only e imutável (ADR-0008 reforça o lado financeiro), registrando ator, ação, tenant, antes/depois quando aplicável.
- **LGPD**: minimização de dados, **dados clínicos fora do MVP** (PRD §3), retenção fiscal de 5 anos (RF-LGPD-02), criptografia de dados pessoais (ADR-0010), e base de tratamento documentada.
- `correlation_id` propagado da requisição pelas filas até os adapters de terceiros.

### CONSEQUÊNCIAS

**Positivas** - Rastreo fim-a-fim de emissão → recebimento → repasse, mesmo assíncrono. - Auditoria e LGPD atendidas por design.

**Negativas / riscos** - Custo de armazenamento/retenção (logs vs auditoria têm políticas distintas). Mitigação: separar retenção de auditoria (5 anos) de logs operacionais (curta). - Cuidado para não logar dado pessoal/sensível — mascarar PII nos logs.

### REFERÊNCIAS

PRD §3, §6, §7.14, §11, RF-LGPD-01..03. Relacionado: ADR-0005, ADR-0008, ADR-0010.

## ADR-0012 · Estratégia de testes e CI no monorepo (pirâmide + contratos + Playwright)

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

### CONTEXTO

Domínio fiscal/financeiro tem baixa tolerância a erro; integrações com terceiros são instáveis (PRD §9, §14); o PRD já adota **Playwright** para e2e. O monorepo (ADR-0001) permite CI por "affected". Invariantes financeiras (ex.: médico sempre recebe 85% — PRD §5.3) pedem verificação rigorosa.

### DRIVERS DE DECISÃO

- Confiança alta no núcleo fiscal/financeiro.
- Detectar quebra de contrato de fornecedor antes da produção.
- Feedback rápido no CI mesmo com o repo crescendo.

### OPÇÕES CONSIDERADAS

1. **Foco em e2e** — lento, frágil, caro; pirâmide invertida.
2. **Pirâmide de testes** (muitos unitários, alguns de integração, poucos e2e) + **testes de contrato** para adapters + **property-based testing (PBT)** para invariantes.

### DECISÃO

- **Pirâmide de testes:**
- **Unitários** do domínio (sem Spring/infra) — núcleo fiscal/ledger;
- **Integração** com PostgreSQL real via **Testcontainers** (inclui testes de **isolamento multi-tenant/RLS** — ADR-0003);
- **e2e** com **Playwright** nos fluxos críticos (onboarding, emissão recorrente, repasse).
- **Testes de contrato** para cada adapter de terceiro (ADR-0006) e **para os contratos entre serviços** (consumer-driven, dado o desenho de microsserviços — ADR-0002), validando o contrato sem depender do serviço externo/par em CI.
- **Property-based testing** para invariantes do motor fiscal e do ledger (ex.: repasse = 85% do bruto; soma débitos = créditos) — ADR-0007/0008.
- **CI no Nx por "affected"**: roda apenas o que mudou, com cache (ADR-0001); gates de cobertura no núcleo crítico e enforcement de boundaries.

### CONSEQUÊNCIAS

**Positivas** - Confiança no núcleo crítico; quebras de contrato pegadas cedo; CI rápido. - PBT cobre casos que exemplos não anteciparam.

**Negativas / riscos** - e2e exige ambiente/seed e cuidado com flakiness. Mitigação: limitar e2e a fluxos críticos, dados determinísticos. - Testcontainers/PBT têm curva inicial. Aceitável dado o domínio.

### REFERÊNCIAS

PRD §5.3, §6, §9, §11, §14. Relacionado: ADR-0001, ADR-0003, ADR-0006, ADR-0007, ADR-0008.

# ADR-0013 · Frontend React e contrato de API (OpenAPI) com tipos gerados no monorepo

**Aceito** · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

## CONTEXTO

O frontend é uma SPA React (portal do médico + backoffice), responsiva, sem app nativo no MVP (PRD §3, §6, §7.12, §7.13). No monorepo (ADR-0001), backend e frontend evoluem juntos; mudanças de contrato devem ser atômicas e tipadas.

## DRIVERS DE DECISÃO

- Contrato de API único e versionado, consumido com segurança de tipos.
- Reaproveitar tipos/cliente entre portal e backoffice.
- Evitar divergência silenciosa entre backend e frontend.

## OPÇÕES CONSIDERADAS

1. **Contrato implícito / tipos duplicados manualmente** — propenso a divergência.
2. **OpenAPI como fonte da verdade + cliente e tipos TS gerados** + lint de contrato no CI.

## DECISÃO

- **OpenAPI** versionado em `contracts/` é a **fonte da verdade** do contrato REST.
- **Cliente + tipos TypeScript gerados** a partir do OpenAPI, consumidos por portal e backoffice (lib compartilhada em `libs/frontend`).
- React com **TypeScript**; design-system/lib de UI compartilhados; estado de servidor com biblioteca de data-fetching (cache/retry) alinhada aos estados assíncronos do backend (ADR-0005).
- **MVP é web responsivo** (sem nativo — PRD §3); estados de processamento ("emitindo", "em conciliação") refletem a natureza assíncrona da esteira.
- Quebra de contrato é detectada no CI (diff de OpenAPI + build do front "affected").

## CONSEQUÊNCIAS

**Positivas** - Segurança de tipos fim-a-fim; mudança de contrato é atômica no monorepo. - Menos bugs de integração front/back.

**Negativas / riscos** - Geração de código exige etapa no build/CI. Mitigação: alvo Nx dedicado. - OpenAPI precisa ser mantido fiel — gerar a partir do código backend ajuda.

## REFERÊNCIAS

PRD §3, §6, §7.12, §7.13. Relacionado: ADR-0001, ADR-0005, ADR-0012.

# ADR-0014 · Modelagem de eventos de domínio e read models (CQRS na leitura)

Proposto · 2026-06-08 · Decisores: Time Pin Saúde (eng. + contabilidade nos pontos fiscais)

## CONTEXTO

Com a **fronteira de dados por serviço** (ADR-0002) — um schema por serviço e **sem acesso cross-schema** — um serviço não consulta os dados do outro diretamente, mesmo na base única do MVP. Os dados que um serviço precisa de outro, e principalmente as **consolidações da gestão** (apuração mensal, DRE, posição de caixa, monitor de teto — PRD §7.9), dependem de **eventos de domínio** (RabbitMQ + Transactional Outbox — ADR-0005) que alimentam **read models**. Para começar certo, o backbone de eventos precisa de convenções claras desde o MVP: o que é publicado, como é versionado e como as projeções de leitura são construídas e reconstruídas. Definir isso já no MVP é o que mantém **barata a futura quebra** em bases por serviço (ADR-0002).

ADR novo (2026-06-08), **pendente de validação do time**. Decorre da fronteira de dados por serviço definida no ADR-0002.

## DRIVERS DE DECISÃO

- Sem JOIN cross-service: precisamos de eventos + read models bem definidos.
- Consolidações da gestão usam dados de fiscal/faturamento/ledger/repassse.
- Evoluir contratos de evento sem quebrar consumidores.
- Rastreabilidade/auditoria de fatos de negócio (PRD §7.14).
- Não acoplar serviços ao modelo interno uns dos outros.

## OPÇÕES CONSIDERADAS

1. **Event notification (eventos "magros", só IDs)** — consumidor precisa "ligar de volta" via API (mais acoplamento síncrono).
2. **Event-carried state transfer (evento "carrega" o estado necessário)** — consumidor projeta sem chamadas de volta; precisa versionar o payload.
3. **Event Sourcing como padrão geral** — poderoso, porém complexo; descartado como padrão no MVP (exceção: o ledger já é append-only/event-like — ADR-0008, mas isso é interno ao serviço).

## DECISÃO

- **Eventos de domínio são contrato de primeira classe**, com **schema versionado** em `contracts/` (ex.: JSON Schema/Avro), **separado do modelo interno** de cada serviço (nunca publicar entidades internas).
- Estilo **event-carried state transfer** para o que outros serviços projetam; **event notification** quando só o gatilho importa. Pragmático: payload com o necessário, sem expor o modelo interno.
- **Envelope padrão** em todo evento: `event_id`, `type`, `version`, `occurred_at`, `cnpj_id` (tenant — ADR-0003), `aggregate_id`, `payload`. Nomes no passado: `NotaEmitida`, `RecebimentoConciliado`, `RepassseEfetuado` ...
- **Entrega confiável e ordem**: Outbox + RabbitMQ (ADR-0005), ordenação por chave de agregado quando necessário, **consumidores idempotentes** (dedupe por `event_id`).
- **Read models (CQRS na leitura)**: o serviço de **gestão** mantém projeções denormalizadas (apuração por CNPJ/competência, DRE, posição de caixa, teto) construídas a partir dos eventos. São **cache derivado** (não fonte da verdade) e **reconstruíveis por replay**.
- **Versionamento**: preferir mudanças retrocompatíveis; quebra → nova `version` / `type`, consumidores tolerantes a campos desconhecidos. Contratos de evento cobertos por **testes de contrato** (ADR-0012).
- **Catálogo de eventos** em `contracts/` como fonte da verdade (publicadores × consumidores).
- **Event Sourcing não é padrão geral**; o ledger permanece append-only internamente (ADR-0008).

## CONSEQUÊNCIAS

**Positivas** - Serviços desacoplados; apuração/teto consolidados **sem JOIN cross-service**. - Read models reconstruíveis (replay) e auditáveis. - Contratos de evento explícitos reduzem quebras entre serviços.

**Negativas / riscos** - **Consistência eventual nas telas de gestão** (a projeção "atrasa" um pouco) — exibir "atualizado em" e desenhar a UX para isso. - **Versionamento de eventos** dá trabalho — mitigar com schemas versionados + testes de contrato. - Risco de **duplicar regra** entre serviço dono e read model — manter o read model **só para leitura/relatório**. - Necessidade de **replay/reprocessamento** — consumidores idempotentes + snapshots das projeções.

## REFERÊNCIAS

PRD §7.9, §7.14, §8, §10. Relacionado: ADR-0002, ADR-0003, ADR-0005, ADR-0008, ADR-0011, ADR-0012.